

Programação Competitiva

Semana 5: Estruturas de dados

Alberto Tavares Duarte Neto

17 de Setembro, 2025

Universidade de Brasília

Estruturas de dados

Definição

Uma **estrutura de dados** é uma maneira de armazenar e organizar dados para facilitar acesso e modificação.

Exemplos de estruturas de dados:

- Vetor.
- Pilha.
- Fila.
- Fila de prioridade.
- Mapa.
- Árvore binária de busca (*set*).

Não existe uma estrutura ótima para todas as situações. Cada estrutura tem restrições e complexidades diferentes.

Definição

Uma **pilha** é uma estrutura onde a remoção remove o elemento mais recentemente inserido.

- Acesso: $O(1)$.
- Inserção: $O(1)$.
- Remoção: $O(1)$.

Pilha

```
stack<int> st;  
st.push(1);  
st.push(2);  
cout << st.top() << endl; // imprime o 2  
st.pop(); // apaga o 2  
st.push(3);  
cout << st.top() << endl; // imprime o 3
```

Definição

Uma **fila** é uma estrutura onde a remoção remove o elemento mais antigo inserido.

- Acesso: $O(1)$.
- Inserção: $O(1)$.
- Remoção: $O(1)$.

Fila

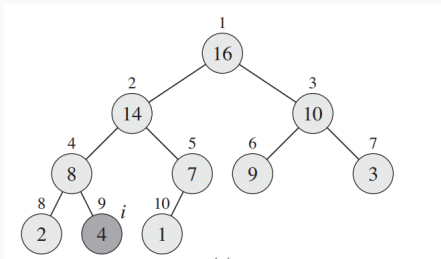
```
queue<int> st;  
st.push(1);  
st.push(2);  
cout << st.front() << endl; // imprime o 1  
st.pop(); // apaga o 1  
st.push(3);  
cout << st.front() << endl; // imprime o 2
```

Fila de prioridade

Definição

Uma **fila de prioridade** é uma estrutura onde a remoção remove o elemento com maior valor.

A implementação da fila de prioridade é feita a partir de uma heap de máximo (cf. Cormen, Cap. 6). Por exemplo, uma heap de máximo armazenando os elementos $\{1, 2, 3, 4, 7, 8, 9, 10, 14, 16\}$ é



Fila de prioridade

A cada inserção ou remoção, é feito um rebalanceamento na árvore para que sua altura seja sempre $\log n$, onde n é a quantidade de elementos na estrutura. Assim, é possível realizar as operações com as seguintes complexidades:

- Acesso: $O(1)$.
- Inserção: $O(\log n)$.
- Remoção: $O(\log n)$.

Fila de prioridade

```
priority_queue<int> st;  
st.push(7);  
st.push(5);  
cout << st.top() << endl; // imprime o 7  
st.pop(); // apaga o 7  
st.push(10);  
cout << st.top() << endl; // imprime o 10
```

Árvore binária de busca

Definição

Seja T uma árvore binária em que cada nó possui um valor. Dizemos que T satisfaz a propriedade de **árvore binária de busca** se, para todo nó x de T , vale:

- Se y é um nó na subárvore da esquerda de x , então $y.valor \leq x.valor$.
- Se y é um nó na subárvore da direita de x , então $y.valor \geq x.valor$.

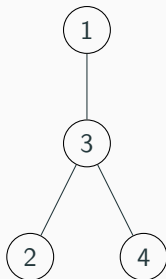
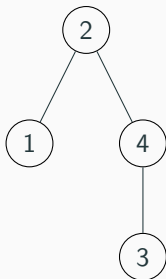
O objetivo é manter a árvore com altura balanceada: se a árvore tem n nós, queremos que sua altura seja $\log n$. Assim, a complexidade das operações é

- Acesso: $O(\log n)$.
- Inserção: $O(\log n)$.
- Remoção: $O(\log n)$.

Árvore binária de busca

Existem algumas técnicas para, após uma inserção ou remoção, rebalancear a árvore com complexidade $O(\log n)$. Veja o capítulo 13 do Cormen.

Para uma mesma lista I , é possível criar várias árvores binárias de busca possuindo nós com os valores de I . Por exemplo, se $I = \{1, 2, 3, 4\}$, então temos



Um *set* em C++ é implementado como uma árvore binária de busca. Temos os seguintes métodos para acessar elementos de um *set*:

- `find(x)`: retorna um iterator para o elemento *x*, se houver, ou `set.end()` caso contrário.
- `lower_bound(x)`: retorna um iterator para o menor elemento maior ou igual a *x*, ou `set.end()` caso contrário.
- `upper_bound(x)`: retorna um iterator para o menor elemento maior que *x*, ou `set.end()` caso contrário.

Para acessar o elemento de um iterator *it*, usamos o operador `*`: `*it`.

Set

```
set<int> s;  
s.insert(1);  
s.insert(3);  
  
auto it1 = s.find(1);  
cout << *it1 << endl; // imprime 1  
auto it2 = s.find(2);  
// acessar *it2 gera erro, pois it2 == s.end()  
  
auto it3 = s.lower_bound(1);  
cout << *it3 << endl; // imprime 1  
  
auto it4 = s.lower_bound(2);  
cout << *it4 << endl; // imprime 3
```